Devoir d'Informatique n°2

2 décembre 2023

*
* *

Durée de l'épreuve : 3 heures

Consignes

Pour répondre à une question, il vous est permis de réutiliser le résultat d'une question antérieure même si vous n'avez pas réussi à établir ce résultat. En particulier, vous pouvez utiliser une fonction demandée dans une question afin de répondre à une question ultérieure.

Vous devrez traiter les questions de programmation dans le langage C. Quand l'énoncé demande de coder une fonction, sauf demande explicite, il n'est pas nécessaire de justifier la complexité, la correction ou la terminaison de cette fonction.

Vous attacherez la plus grande importance à la clarté et à la précision de la rédaction.

Si vous repérez ce qu'il vous semble être une erreur d'énoncé, signalez-la sur votre copie et poursuivez la composition en expliquant les éventuelles initiatives que vous aurez pris.

Exercices

Exercice 1.

Q1. Pour $u = (u_n)_{n\in\mathbb{N}} \in (\mathbb{R}^+)^{\mathbb{N}}$ une suite, rappeler la définition des ensembles $\mathcal{O}(u)$ et $\Theta(u)$.

Q2. Expliquer comment s'organise la mémoire dans le tas.

Q3. Écrire une fonction `char* strpyco(char* dst, char* src)` qui recopie la chaîne de caractères `src` dans `dst`, dans l'ordre inverse. Par exemple :

```
char* s = "blabla";
char t[20];
strpyco(t, s); // t contient "albalb";
```

Vous n'avez pas le droit d'utiliser les fonctions de la la librairie `<string.h>`.

Q4. Écrire du code C permettant de définir un type `struct stylo` ayant pour attributs :

- Une chaîne de caractères donnant la couleur du stylo ;
- Un flottant donnant le pourcentage d'encre restant ;
- Un booléen disant si le stylo est effaçable.

Écrire également du code permettant de définir une variable de type `struct stylo*` pointant vers le tas, vers un tableau de deux structures représentant :

- un stylo vert effaçable à moitié plein.
- un stylo rouge non-effaçable vide.

Exercice 2.

Q1. On fixe u, v, w, z des suites à valeurs dans R^+. Pour chacune des affirmations ci dessous, indiquer si elle est vraie ou fausse. Si elle est vraie, donner une preuve ; si elle est fausse, donner un contre-exemple.

a) Si $u_n = \mathcal{O}(w_n)$ et $v_n = \mathcal{O}(w_n)$, alors $u_n + v_n = \mathcal{O}(w_n)$.
b) Si $u_n = \mathcal{O}(w_n)$ et $v_n = \Omega(w_n)$ alors $u_n + v_n = \Theta(w_n)$.
c) Si $u_n = \Theta(w_n)$ et $u_n = \mathcal{O}(w_n)$ alors $u_n + v_n = \Theta(w_n)$.
d) Si $u_n = \mathcal{O}(v_n)$, alors $u_n \leq v_n$ à partir d'un certain rang

Q2. Écrire une fonction `int** produits(int n)` qui renvoie un tableau 2D d'entiers, de taille $n \times n$, où pour $i, j \in [\![0, n-1]\!]$, la case (i, j) contient le produit $\prod_{k=i}^{j} k$. On rappelle qu'un produit vide (par exemple $\prod_{k=10}^{3} k$) vaut 1. Votre fonction devra être en $\mathcal{O}(n^2)$, mais vous n'avez pas besoin de justifier cette complexité.

Exercice 3.

La mairie de Grainville-sur-Odon souhaite mettre au point un logiciel pour gérer les nombreux festivals et autres évènements qui s'y tiennent. Pour cela, on s'intéresse à l'implémentation d'une librairie C simple, manipulant le type suivant :

```c
struct festival {
   char nom[40];
   int d; // début
   int f; // fin
};
typedef struct festival fes_t;
```

Les attributs `.d` et `.f` sont les dates de début et de fin d'un festival, dans une unité arbitraire, et signifient que le festival couvre les dates $d, d+1, \ldots f$, intervalle que l'on notera $[\![d, f]\!]$ Par convention, on considèrera que tout intervalle où $d > f$ est **vide** : un tel intervalle ne devrait raisonnablement jamais apparaître dans le produit final, mais les fonctions que l'on implémente devront bien les gérer.

Q1. On souhaite stocker le code de la librairie dans deux fichiers `festival.h` et `festival.c`. On rappelle l'existence des instructions préprocesseur `#ifndef`, `#endif` et `#define` :

— Tout code compris entre un `#ifndef NOM_MACRO` et un `#endif` sera lu par le compilateur si et seulement si la macro `NOM_MACRO` n'a pas été définie

— La commande `#define NOM_MACRO` permet de définir la macro `NOM_MACRO` (sans lui donner de valeur particulière).

A l'aide de ces macros, expliquer comment implémenter un *include guard* pour le fichier `intervalle.h`, et préciser son utilité.

Q2. Écrire une fonction `bool contient(fes_t* A, int x)` déterminant si le jour x tombe pendant le festival A.

Q3. Écrire une fonction `bool intersecte(fes_t* A, fes_t* B)` déterminant si les festivals A et B ont un jour en commun.

Le fichier `festivals.txt` (Fig. 1) contient les informations des différents festivals se tenant dans la ville. Chaque ligne du fichier stocke les informations d'un festival : le nom, la date de début, la date de fin. On pourra supposer que les noms des festivals ne contiennent pas d'espace, et qu'ils font moins de 40 caractères.

```
Salon_de_la_poterie 5 11
80_eme_anniversaire_liberation 8 15
Coachella 12 20
...
```

FIGURE 1 – Fichier `festival.txt`

L'annexe B du sujet contient des rappels sur l'utilisation des fonctions `fopen`, `fclose` et `fscanf`.

Q4. Écrire une fonction `int lire_festivals(char* nom_fichier, fes_t* festivals, int n_max)` qui lit dans le fichier passé en argument, supposé être au même format que `festivals.txt` et stocke les informations lues dans le tableau `festivals`. Le paramètre `n_max` donne la taille allouée pour `festivals`, et la fonction n'a donc pas le droit de lire plus que `n_max` entiers. La fonction renverra le nombre de festivals lus.

On note N le nombre total de festivals à organiser. La ville possède K terrains numérotés de 0 à $K-1$, pouvant être utilisés pour héberger les différents évènements. Deux évènements ne peuvent être organisés sur le même terrain que s'ils ne s'intersectent pas, et la mairie doit donc trouver une affection acceptable des terrains aux différents évènements. Une affectation sera donnée par un tableau de N cases, la i-ème donnant l'indice du terrain affecté à l'évènement i.

Q5. Écrire une fonction ayant la spécification suivante, en complexité $\mathcal{O}(N^2)$.

```
/* Renvoie true si l'affectation donné par `terrains` est valide.
   Paramètres:
   - festivals: tableaux des festivals
   - terrains: tableau des terrains affectés.
               terrains[i] contient le terrain affectés au festival i.
   - N: taille du tableau `festivals`
   - K: nombre de terrains */
bool verifie_affectation(fes_t* festivals, int* terrains, int N, int K);
```

Q6. Écrire une fonction `int* affectation_simple(fes_t* festivals, int N, int K)` essayant de trouver une affectation valide des K terrains aux N évènements, en suivant l'algorithme suivant :

---

pour $i = 0$ **à** $N-1$ **faire**
  $j \leftarrow$ indice minimal d'un terrain pouvant être affecté à l'évènement i;
  Affecter à l'évènement i le terrain j;

La fonction renverra l'affectation créée, ou bien le pointeur `NULL` si l'algorithme échoue, c'est à dire s'il faudrait strictement plus que K terrains pour l'appliquer intégralement. Quelle est sa complexité ?

Problème : Langage machine

Langage machine, processeur

Les programmes compilés, comme tous les fichiers, sont des suites d'octets. Ces fichiers ne sont pas a priori lisibles facilement par les humains, mais ils suivent bien une structure : là où un fichier C est écrit dans le langage C, un programme compilé est écrit dans un **langage machine**. Il existe de nombreuses familles de langages machines : ARM (sur la plupart des téléphones), x86 (sur beaucoup d'ordinateurs), RISC-V, etc... Chaque langage machine possède un **jeu d'instruction**, qui est l'ensemble des instructions existantes du langage.

Lorsque l'on lance un programme, ses instructions sont lues et exécutées par le **processeur**, un composant électronique de l'ordinateur. Le processeur est capable d'exécuter des millions (même des milliards) d'instructions par seconde : calculs, accès à la mémoire RAM, etc... Tous les programmes qui tournent sur un ordinateur sont exécutés par le processeur.

L'objectif de ce problème est d'étudier un jeu d'instruction simple, inspiré du RISC-V, et d'implémenter en C un émulateur, un logiciel permettant de simuler l'exécution de programmes écrits dans ce langage.

Instructions et opcode

Le langage machine MPI (**M**es **P**etites **I**nstructions) utilise des instructions tenant sur 16 bits. Une instruction MPI est constituée de 4 bits d'**opcode** et de 12 bits d'**opérandes**. L'*opcode*, ou code d'opération, permet de savoir la nature de l'instruction : une addition, une lecture / écriture mémoire, la vérification d'une condition, etc... Les opérandes sont les arguments de l'instruction.

Q1. Combien d'instructions distinctes (i.e. d'*opcodes* distincts) peut-on avoir dans le langage MPI ?

Un tableau contenant les différentes instructions du langage MPI est fourni dans l'annexe A du sujet. Il est conseillé de ne pas le lire immédiatement, mais de s'y référer au fur et à mesure que les différentes instructions sont introduites.

Registres

Le processeur possède une petite mémoire appelée les **registres**, qu'il peut utiliser pour faire des calculs. Les instructions permettent de modifier les valeurs des registres, en effectuant des calculs. Pour les machines MPI, on suppose que l'on dispose de 16 registres, appelés $r_0, \dots r_{15}$, pouvant chacun contenir 8 bits d'information.

Le registre r_0 est un **registre nul** : sa valeur est toujours 0, et il est impossible d'y écrire autre chose.

Première instruction : l'addition

Lorsque l'*opcode* d'une instruction est `0000`, l'instruction est une addition d'entiers signés. Elle prend alors la forme :

$$\underbrace{0000}_{\text{opcode}}\ \underbrace{d_3d_2d_1d_0}_{\text{destination}}\ \underbrace{x_3x_2x_1x_0}_{\text{opérande 1}}\ \underbrace{y_3y_2y_1y_0}_{\text{opérande 2}}$$

Les bits $d_3, \dots d_0, y_3, \dots, y_0, x_3 \dots x_0$ sont les écritures binaires de numéros de registre. Par exemple, l'instruction `0000 0101 0010 1001` a pour effet de stocker dans le registre 5 ($= \overline{0101}^2$) la somme des registres 2 ($= \overline{0010}^2$) et 9 ($= \overline{1001}^2$). Ainsi, si le registre 2 contient l'octet `0x46` ($= 70$) et que le registre 9 contient `0x18` ($= 24$) avant l'instruction, alors après l'instruction, le registre 5 contiendra l'octet `0x5d` ($= 94$). En hexadécimal, cette instruction s'écrira `0x0529` :

```
0000 0101 0010 1001
0    5    2    9
```

Q2. On suppose que les registres contiennent les valeurs suivantes :

Registre n°	0	1	2	3	4	5	6	7
Octet stocké	`0x00`	`0x13`	`0xa7`	`0x2d`	`0x44`	`0x60`	`0x00`	`0x14`
Registre n°	8	9	10	11	12	13	14	15
Octet stocké	`0xe3`	`0xe2`	`0x21`	`0x17`	`0x03`	`0x41`	`0x40`	`0xff`

Décrire l'état des registres après avoir exécuté la suite d'instructions suivante :

```
0x017a
0x09de
0x0819
0x0333
```

Montrez votre raisonnement et vos calculs.

Les *opcodes* `0001` et `0010` correspondent respectivement à la soustraction et à la multiplication. Ils fonctionnent exactement comme l'addition : l'*opcode* est suivi du registre de destination puis des deux registres d'opérande.

Mnémonique

Afin de faciliter la lecture et l'écriture de code machine, on utilise parfois des **mnémoniques**, qui sont des symboles représentant les différentes instructions. Pour l'addition, la soustraction et la multiplication, on utilise les symboles ADD, SUB et MUL, suivi des registres d'opérande. A titre d'exemple, considérons la suite de 3 instructions ci-dessous, où chaque instruction est notée en hexadécimal et en mnémonique, et est accompagnée d'un commentaire la décrivant :

```
INSTRUCTION  MNEMONIQUE       // DESCRIPTION
0x017a       ADD r1  r7  r10  // r1 reçoit r7 + r10
0x19de       SUB r9  r13 r14  // r9 reçoit r13 - r14
0x2819       MUL r8  r1  r9   // r8 reçoit r1 * r9
0x0333       ADD r3  r3  r3   // r3 reçoit r3 + r3
```

Q3. Traduire en hexadécimal la suite de mnémoniques suivante :

```
ADD r8 r0 r15
MUL r9 r9 r3
SUB r12 r11 r3
```

Dans la suite, lorsque vous devrez écrire du code MPI, sauf si la question le précise autrement, vous pourrez donner votre réponse sous la forme de suite de mnémoniques plutôt qu'en hexadécimal.

Spécification

La **spécification** d'un programme MPI est une description formelle de ce que fait le code du programme. On donne la spécification d'un programme en précisant :

— des préconditions, c'est à dire des propriétés devant être vérifiées pour que le code s'exécute correctement ;

— des postconditions, c'est à dire des propriétés étant vraies si le code est exécuté dans le cadre des préconditions.

Par exemple, voici un code MPI commenté, et précédé de sa spécification :

```
Précondition: r3 contient un entier x quelconque
Postcondition: r4 contient l'entier 4x
0x0433 ADD r4 r3 r3 // r4 reçoit r3 + r3 = 2x
0x0444 ADD r4 r4 r4 // r4 reçoit r4 + r4 = 2x + 2x = 4x
```

Q4. On rappelle que le registre r_0 est toujours nul. Écrire une instruction MPI vérifiant la spécification suivante :

```
Précondition: r1 contient un entier x quelconque
Postcondition: r2 contient x
```

Q5. Écrire du code MPI vérifiant la spécification suivante :

```
Précondition: r1 contient un entier x quelconque,
r2 contient un entier y quelconque
Postcondition: r1 contient y et r2 contient x.
Les registres r4, ..., r15 n'ont pas été modifiées.
```

Constante immédiate

L'instruction de constante immédiate permet de stocker une constante dans un registre donné. Son *opcode* est 0100, et elle s'écrit :

$$\underbrace{0100}_{\text{opcode}}\,\underbrace{d_3d_2d_1d_0}_{\text{destination}}\,\underbrace{x_7x_6x_5x_4x_3x_2x_1x_0}_{\text{constante}}$$

Par exemple, après l'instruction `0100 0101 0010 1010`, le registre 5 contient l'octet `0x2a`. La notation mnémonique pour cette instruction est `IMM r5 42` (car 42 s'écrit `0x2a`).

Q6. Écrire les instructions suivantes en hexadécimal, et donner l'état des registres après leur exécution, en partant de registres tous nuls.

```
IMM r1 1
ADD r2 r2 r1
ADD r3 r3 r2
ADD r2 r2 r1
ADD r3 r3 r2
ADD r2 r2 r1
ADD r3 r3 r2
```

Compteur de programme, saut inconditionnel

On rappelle que la **RAM** désigne la mémoire utilisée par un programme lors de son exécution, c'est un immense tableau d'octets numérotés à partir de 0. Lorsque l'on exécute un programme compilé, l'intégralité de son code est recopié dans la RAM, où le processeur peut lire les instructions une à une et les exécuter. Pour simplifier, on supposera dans la suite que les instructions sont toujours écrites à partir de l'adresse zéro. **Attention**, comme nos instructions font 16 bits, soit 2 octets, les instructions se trouveront toujours à des adresses multiples de 2 : La première instruction sera à l'adresse 0, la suivante à l'adresse 2, et ainsi de suite.

Le processeur possède un registre particulier, appelé le compteur de programme ou **program counter**, et souvent noté PC. Il stocke l'adresse de la **prochaine instruction à exécuter**. Au lancement, le PC vaut 0, puis après avoir exécuté la première instruction, il passe à 2, puis à 4, et ainsi de suite. Dans la suite, dans les programmes MPI qui vous seront donnés, les adresses des instructions seront spécifiées, en hexadécimal.

Certaines instructions permettent de modifier le PC, et donc de faire des **sauts** dans le code. C'est ce mécanisme qui va permettre de faire des boucles.
Le **saut inconditionnel** est un exemple d'une telle instruction. Il prend la forme :

$$\underbrace{1011}_{\text{opcode}}\ \underbrace{d_3d_2d_1d_0}_{\text{registre d'adresse}}\ \underbrace{00000000}_{\text{inutilisé}}$$

Le seul opérande, le registre d'adresse, est le numéro du registre dans lequel lire l'adresse de la prochaine instruction à exécuter. Le symbole mnémonique utilisé est JMP.

Par exemple, si l'on considère le programme suivant :

```
Addr Instruction Mnémonique
0000 0x4346      IMM r3 70
0002 0xb300      JMP r3
0004 0x4607      IMM r6 7
...
```

La première instruction stocke la valeur 70 dans r_3. La deuxième instruction est un saut inconditionnel : après cette instruction, au lieu d'incrémenter le PC de 2 et d'exécuter l'instruction suivante, le PC va passer à 70, et le programme va continuer à s'exécuter à partir de l'instruction d'adresse 70, c'est à dire la 36-ème instruction du programme.

Q7. On considère le programme suivant, exécuté à partir de registres tous nuls. Déterminer les valeurs successives prises par les registres r_2 et r_1, en justifiant :

```
Addr Instruction Mnémonique
0000 0x4101      IMM r1 1
0002 0x4202      IMM r2 2
0004 0x4301      IMM r3 1
0006 0x4708      IMM r7 8
0008 0x2112      MUL r1 r1 r2
000a 0x0223      ADD r2 r2 r3
000c 0xc700      JMP r7
```

Q8. Écrire du code MPI tournant à l'infini, et faisant prendre à r_2 les valeurs $0, 1, 4, 9, 16, \ldots$ (les carrés parfaits). On ignorera les problèmes de dépassement d'entier. Expliquez brièvement votre code, notamment le(s) saut(s) que vous aurez utilisé(s).

Saut conditionnel

Les sauts inconditionnels ne suffisent pas pour pouvoir simuler des boucles “tant que”. Les instructions de **sauts conditionnels** permettent d'effectuer un saut uniquement lorsqu'une condition donnée est vérifiée. La forme général du saut conditionnel est :

$$\underbrace{11XX}_{\text{opcode}} \quad \underbrace{d_3d_2d_1d_0}_{\text{registre d'adresse}} \quad \underbrace{x_3x_2x_1x_0}_{\text{opérande 1}} \underbrace{y_3y_2y_1y_0}_{\text{opérande 2}}$$

Notons r_d le registre d'adresse et r_x et r_y les registres opérandes. Le registre r_d contient l'adresse à laquelle effectuer le saut, et les registres r_x et r_y servent à former la condition de saut. Les différentes conditions possibles sont notées dans le tableau ci-contre.

Opcode	Mnémonique	Condition
1100	`JEQ rd rx ry`	Si $r_x = r_y$
1101	`JNE rd rx ry`	Si $r_x \neq r_y$
1110	`JLE rd rx ry`	Si $r_x \leq r_y$
1111	`JLT rd rx ry`	Si $r_x < r_y$

Par exemple, considérons le code suivant :

```
Addr Instruction Mnémnonique
0000 0x4101   IMM r1 1
0002 0x4202   IMM r2 2
0004 0x43c2   IMM r3 194
0006 0xd700   JNE r3 r2 r1
```

Les trois premières instructions stockent les valeurs 1, 2 et 194 dans les registres r_1, r_2, r_3. Puis, la dernière instruction regarde si $r_2 \neq r_1$, et comme c'est le cas, le programme exécute ensuite l'instruction d'adresse 194.

Q9. Exécuter le programme suivant à partir de registres nuls, sauf r_4 qui vaut 5. Donner la valeur de r_3 à la fin de l'exécution, et proposer une spécification convenable.

```
Addr Instruction Mnémonique
0000 0x4101      IMM r1 1
0004 0x4708      IMM r7 8
0006 0x4810      IMM r8 16
0008 0xc824      JEQ r8 r2 r4
000a 0x0332      ADD r3 r3 r2
000c 0x0221      ADD r2 r2 r1
000e 0xb700      JMP r7
0010 0x0110      ADD r1 r1 r0
```

Q10. Proposer un algorithme en pseudo-code permettant d'effectuer une division euclidienne, en n'utilisant que des soustractions et des additions :

Algorithme 1 : Division

Entrée(s) : $a, b \in \mathbb{N}$ avec $b \neq 0$

Sortie(s) : Quotient et reste de la division euclidienne de a par b

Q11. En s'inspirant de cet algorithme, écrire du code MPI vérifiant la spécification suivante

```
Précondition: r1 et r2 contiennent des entiers quelconques, avec r2 non nul
Postcondition: r1 et r2 n'ont pas été changés. r3 contient le quotient
de la division euclidienne de r1 par r2, et r4 contient le reste.
```

Q12. Écrire du code MPI vérifiant la spécification suivante :

```
Précondition: r1 contient un entier strictement supérieur à 1.
Postcondition: r1 n'a pas été changé, et r2 contient le
plus petit diviseur strictement supérieur à 1 de r1.
```

Q13. Expliquer pourquoi les sauts conditionnels rendent l'instruction `JMP` redondante.

Émulation en C

On s'intéresse à l'écriture d'un programme C permettant de lire des fichiers exécutables dans le langage MPI, et de simuler leur exécution. On appellera ce programme un **émulateur**.
On introduit les types C suivants :

```
1 typedef uint16_t instr_t; // instructions
2 typedef int8_t val_t; // valeurs des registres
```

On rappelle que la première ligne signifie que dans la suite, `instr_t` sera un synonyme de `uint16_t`, i.e. des entiers 16 bits. On utilisera ce type pour représenter les instructions, en lisant une instruction comme un nombre écrit en base 2. De même, le type `val_t` permettra de représenter le contenu des registres.

Q14. Écrire une fonction `void bin_instruction(instr_t i)` qui affiche l'instruction donnée en paramètre en binaire, et en séparant en 4 groupes de 4 bits. Par exemple, si l'on appelle `bin_instruction(587)`, la fonction affichera `0000 0010 0100 1011`.

Le programme aura plusieurs variables globales suivantes :

```
1 unsigned int program_size; // nombre d'instructions
2 unsigned int program_counter; // prochaine instruction à exécuter
3 instr_t program[256]; // tableau des instructions
4 val_t reg[16]; // tableau des registres
```

Notons que l'on ne peut traiter que des programmes d'au plus 256 instructions.
Attention, ici le *program counter* n'est pas une adresse mémoire, mais l'indice d'une instruction : après une instruction autre qu'un saut, le *program counter* est incrémenté de 1 dans l'émulateur (et pas de 2).
L'objectif de cette partie est d'implémenter une fonction `void execute(instr_t ins)` qui exécute l'instruction `ins`, et modifie les valeurs des registres ainsi que le *program counter* en conséquence.

Q15. Justifier que si l'on dispose d'une variable `instr_t ins` représentant une instruction, la formule `(ins >> 12) & 0xf` donne l'*opcode* de cette instruction.

Q16. Compléter le code suivant, qui gère le cas où l'instruction est une opération de constante immédiate :

```
1  /* Exécute ins lorsque c'est une opération arithmétique, et modifie
2     le tableau reg en conséquence. */
3  void execute_imm(instr_t ins){
4     // opération de constante immédiate: opcode destination valeur
5     int op = (ins >> 12) & 0xf;
6     assert(op == 4);
7     int dst  =  [A COMPLETER 1];
8     val_t valeur =  [A COMPLETER 2];
9     // modifier le registre
10    reg[dst] = valeur;
11    // augmenter le PC
12    program_counter = program_counter + 1;
13 }
```

Q17. Écrire une fonction `void execute_arith(instr_t ins)` qui exécute l'instruction `ins`, en ayant comme précondition que c'est bien une instruction arithmétique (addition, soustraction, multiplication).

Q18. Écrire une fonction `void execute_jump_uncond(instr_t ins)` qui exécute une instruction de saut inconditionnel.

Q19. On suppose que l'on dispose également d'une fonction `void execute_jump_cond(instr_t ins)` permettant d'exécuter les instructions de sauts conditionnels.
Écrire une fonction `void execute(instr_t ins)` permettant d'exécuter l'instruction `ins` quel que soit son opcode, excepté 1000 et 1001.

Q20. On suppose que l'on dispose d'une fonction

```
bool read2(FILE* f, instr_t* ins);
```

qui lit une instruction (donc deux octets) dans le fichier `f` et stocke le résultat dans `*ins`. La fonction renvoie un booléen : vrai si la lecture a réussi, et faux si la fin du fichier a été atteinte lors de la lecture. Écrire une fonction `void load(char* filename)` qui lit dans le fichier donné en argument et charge en mémoire le programme lu, en modifiant les variables globales `program` et `program_size`.

On rappelle que la fonction `main` d'un programme C peut prendre deux paramètres :

```
int main(int argc, char** argv);
```

Dans ce cas, `argc` donne le nombre d'arguments ayant été utilisés à l'exécution du programme dans le terminal (nom de l'exécutable compris), et `argv` est un tableau de longueur `argc` contenant les différents arguments. En particulier, `argv[0]` contient toujours le nom de l'exécutable, et l'on ne doit jamais accéder à une case `argv[i]` si $i \geq$ **argc**

Q21. Écrire la fonction `main` de l'émulateur. Le programme devra :

1. prendre en argument un nom de fichier directement via ligne de commande, et s'arrêter avec un message d'erreur si l'utilisateur n'a pas fourni de nom de fichier
2. charger un programme à partir du fichier, et s'arrêter avec un message d'erreur si le fichier n'existe pas ;
3. exécuter le programme à partir de registres tous nuls ;
4. afficher le contenu des 16 registres à la fin de l'exécution, en tant qu'entiers non signés.

Par exemple :

```
fredfrigo@ubuntu~$ ./emulator test.mp2
Registres après l'exécution:
R0 = 0
R1 = 245
R2 = 70
...
```

Q22. Quelle commande écrire dans le terminal pour exécuter l'émulateur sur un fichier `test.mp2` et stocker l'affichage de l'émulateur dans un fichier `log.txt` ?

Accès mémoire

Les 16 registres ne permettent pas d'écrire des programmes poussés, car on est limités à 15 variables au total (le registre r_0 étant toujours nul). La plupart de la mémoire d'un programme est stockée non pas dans les registres mais dans la RAM.

Le langage MPI possède deux instructions permettant de lire et d'écrire dans la RAM :

— L'instruction 1000 $d_3d_2d_1d_0$ $l_3l_2l_1l_0$ $u_3u_2u_1u_0$, de mnémonique `LOAD rd rl ru`, lit dans la RAM et stocke l'octet lu dans le registre r_d. L'adresse dans laquelle lire est obtenue en considérant r_l comme les 8 bits de poids faible et r_u comme les 8 bits de poids fort (les indices l et u viennent de l'anglais **l**ower et **u**pper).

— L'instruction 1001 $d_3d_2d_1d_0$ $l_3l_2l_1l_0$ $u_3u_2u_1u_0$, de mnémonique `STORE rd rl ru`, écrit dans la RAM l'octet contenu dans le registre r_d. L'adresse dans laquelle écrire est obtenue de la même manière que pour l'instruction `LOAD`.

Par exemple :

```
Addr Instruction Mnémnonique
0000 0x4135      IMM  r1 53
0002 0x420c      IMM  r2 12
0004 0x8312      LOAD r3 r1 r2
```

Q23. A quelle adresse de la RAM les instructions précédentes vont-elles permettre d'accéder ? Quelle est la taille maximale d'une RAM pouvant être utilisée avec les instructions LOAD et STORE ?

Q24. Écrire du code MPI respectant la spécification suivante :

```
Précondition: r14 et r15 contiennent respectivement les 8 bits de poids
faible et fort d'une adresse mémoire p.
Postcondition: la valeur à l'adresse $p$ a été incrémentée de 1.
```

Q25. Écrire du code MPI respectant la spécification suivante :

```
Précondition: r14 et r15 contiennent respectivement les 8 bits de poids
faible et fort d'une adresse mémoire p. r1 contient un entier n quelconque.
Postcondition: Les valeurs aux adresses p, p+1, ... p+(n-1) ont toutes
été incrémentées de 1.
```

Les traces de recherche seront évaluées. Par exemple si vous écrivez des blocs de code que vous estimez utiles, expliquez-les même si vous n'arrivez pas à tout regrouper en un programme complet.

Un compilateur comme GCC traduit du code C en code machine et, parmi les nombreuses tâches qu'il doit réaliser, il doit décider d'où stocker les variables. Une variable peut être stockée dans la RAM, auquel cas elles sont chargées dans les registres avec LOAD lorsqu'elles doivent être manipulées, et stockées avec STORE. Cependant, les accès mémoire réalisés par LOAD et STORE sont longs. On peut stocker des variables directement dans les registres, ce qui permet d'y accéder bien plus vite. C'est notamment avantageux pour les variables qui sont lues et modifiées très souvent, comme les indices des boucles pour. Les registres étant peu nombreux, il faut choisir avec précaution quelles variables y stocker. Lorsque l'on compile un programme avec GCC, par défaut, toutes les variables sont stockées dans la RAM, les variables locales étant alors notamment stockées dans la pile d'appel. Il existe des options permettant de demander à GCC d'optimiser le programme, ce qui a entre autre pour effet de stocker certaines valeurs dans les registres. Les compilateurs modernes sont très efficaces pour trouver de bonnes stratégies de stockage des variables. Néanmoins, en C, le mot clé `register` *permet de conseiller au compilateur de stocker une variable dans les registres !*

* *
*

Annexe A : Jeu d'instruction MPI

Binaire	Mnémonique	Effet
$0000x_3x_2x_1x_0y_3y_2y_1y_0z_3z_2z_1z_0$	`ADD rx ry rz`	Stocke $r_y + r_z$ dans r_x
$0001x_3x_2x_1x_0y_3y_2y_1y_0z_3z_2z_1z_0$	`SUB rx ry rz`	Stocke $r_y - r_z$ dans r_x
$0010x_3x_2x_1x_0y_3y_2y_1y_0z_3z_2z_1z_0$	`MUL rx ry rz`	Stocke $r_y \times r_z$ dans r_x
0011		Inutilisée
$0100d_3d_2d_1d_0c_7c_6c_5c_4c_3c_2c_1c_0$	IMM r_d c	Stocke c dans r_d
0101		Inutilisée
0110		Inutilisée
0111		Inutilisée
$1000d_3d_2d_1d_0l_3l_2l_1l_0u_3u_2u_1u_0$	`LOAD rd rl ru`	Charge dans r_d le contenu de l'adresse r_ur_l
$1001d_3d_2d_1d_0l_3l_2l_1l_0u_3u_2u_1u_0$	`STORE rd rl ru`	Écrit à l'adresse r_ur_l le contenu de r_d
1010		Inutilisée
$1011a_3a_2a_1a_00000000$	JMP r_a	Saute à l'instruction d'adresse r_a
$1100a_3a_2a_1a_0x_3x_2x_1x_0y_3y_2y_1y_0$	JEQ r_a r_x r_y	Saute à l'instruction d'adresse r_a si $r_x = r_y$
$1101a_3a_2a_1a_0x_3x_2x_1x_0y_3y_2y_1y_0$	JNE r_a r_x r_y	Saute à l'instruction d'adresse r_a si $r_x \neq r_y$
$1110a_3a_2a_1a_0x_3x_2x_1x_0y_3y_2y_1y_0$	JLE r_a r_x r_y	Saute à l'instruction d'adresse r_a si $r_x \leq r_y$
$1111a_3a_2a_1a_0x_3x_2x_1x_0y_3y_2y_1y_0$	JLT r_a r_x r_y	Saute à l'instruction d'adresse r_a si $r_x < r_y$

Annexe B : Fichiers

Les 4 fonctions suivantes permettent de manipuler les fichiers en C :

- `FILE* fopen(char* nom_fichier, char* mode)` permet d'ouvrir le fichier dont le nom est donné en paramètre, dans le mode spécifié ("r" pour la lecture, "w" pour l'écriture, "a" pour l'ajout). Elle renvoie le pointeur nul en cas d'échec d'ouverture, notamment s'il n'existe pas de fichier portant le nom demandé.
- `void fclose(FILE* f)` ferme le fichier donné en paramètre.
- `int fscanf(FILE* f, char* format, ...)` permet de lire dans le fichier `f` selon le même principe que `scanf`. Cette fonction renvoie le nombre d'objets lus, ou bien renvoie la valeur spécial `EOF` si la tête de lecture est à la fin du fichier.
- `void fprintf(FILE* f, char* format, ...)` permet d'écrire dans le fichier `f` selon le même principe que `printf`.

On rappelle également quelques formats de lecture/écriture :

- `"%d"` permet de lire ou d'écrire un `int` ;
- `"%d"` permet de lire ou d'écrire un `unsigned int` ;
- `"%f"` permet de lire ou d'écrire un `float` ;
- `"%s"` permet de lire ou d'écrire un `char*`, mais s'arrête aux espaces et aux sauts de lignes lors d'une lecture.